

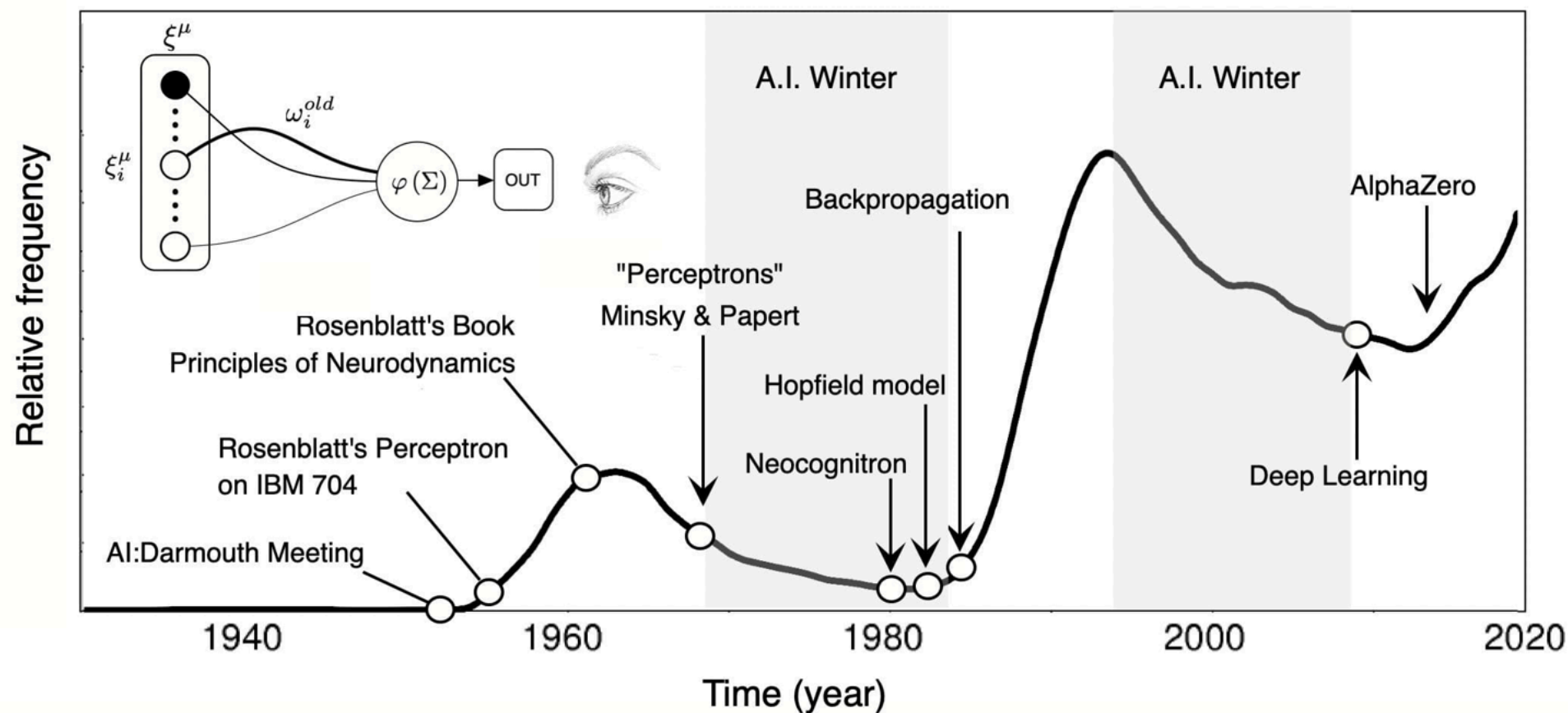
# Neural Networks 2

Hidden Layers and Backpropagation

# Where we are

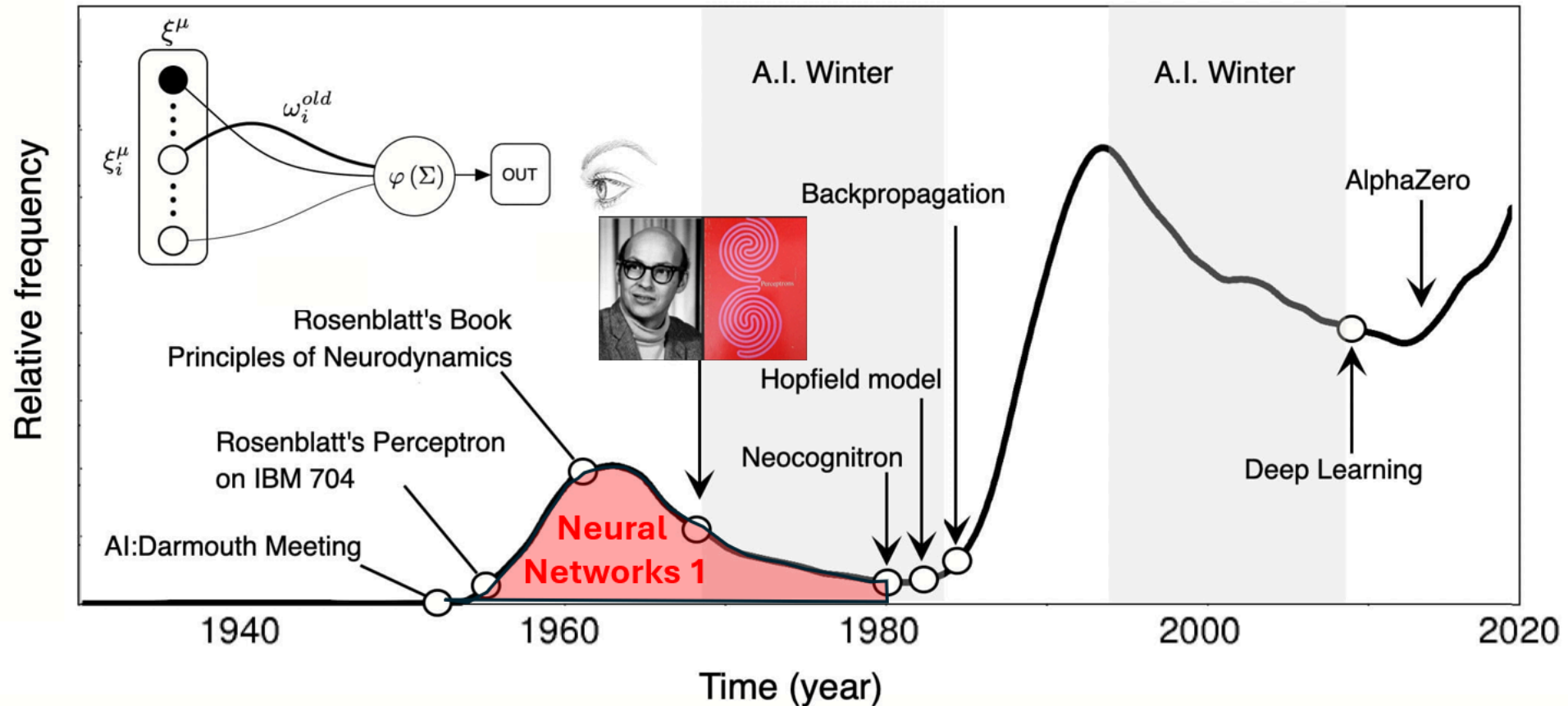
- Previously: a single unit (a perceptron), trained with the delta rule cannot properly resolve the XOR problem. The tutorial had you train one yourself and watch it get stuck, and it's easy to see why: no single straight line separates those four points
- Today: give the network a **hidden layer**, and learn the one new idea — **backpropagation** — needed to train it
- By the end: your network solves XOR, and you'll see exactly how the fix works

# The springs and winters of AI development



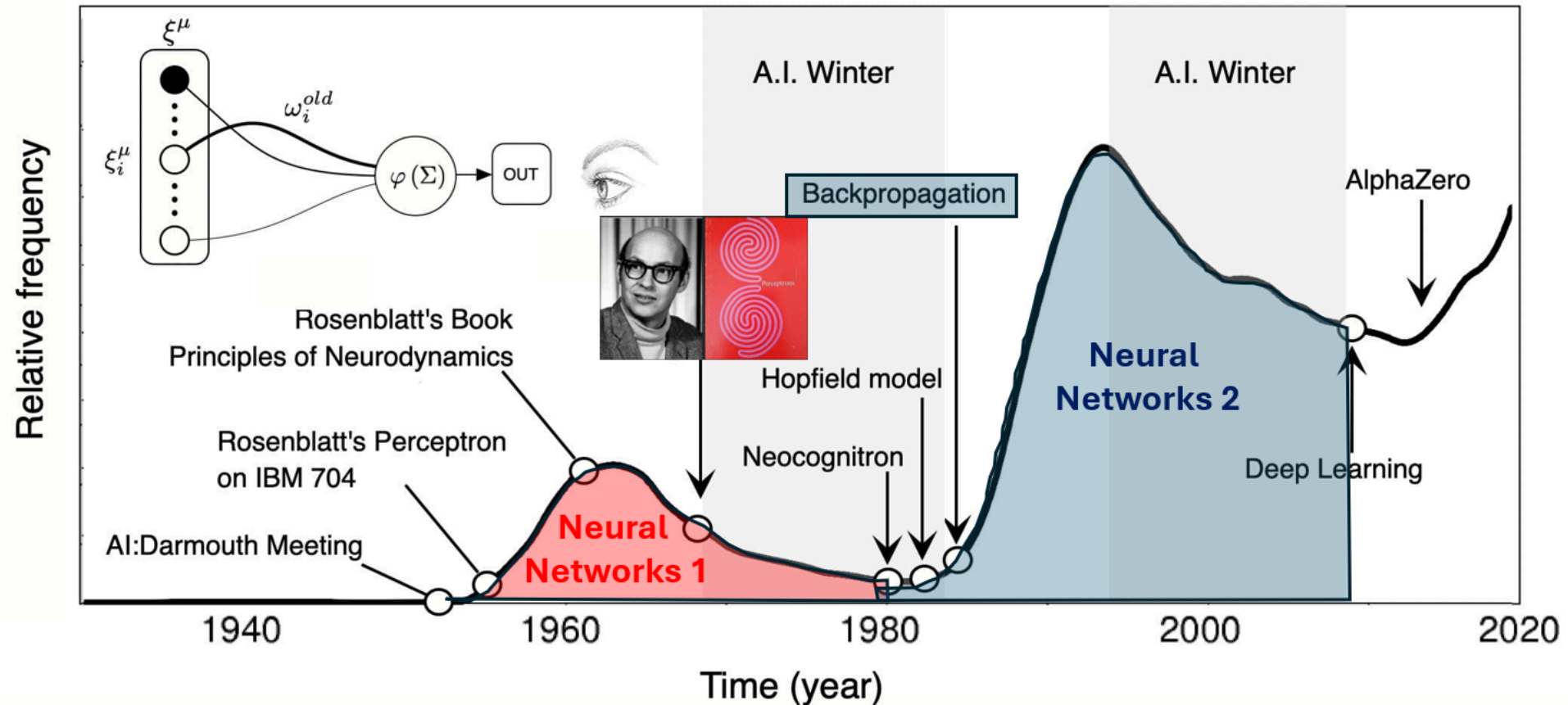
[https://x.com/ricard\\_sole/status/1497306702181548039](https://x.com/ricard_sole/status/1497306702181548039)

# The springs and winters of AI development



[https://x.com/ricard\\_sole/status/1497306702181548039](https://x.com/ricard_sole/status/1497306702181548039)

# The springs and winters of AI development



[https://x.com/ricard\\_sole/status/1497306702181548039](https://x.com/ricard_sole/status/1497306702181548039)

# Learning objectives

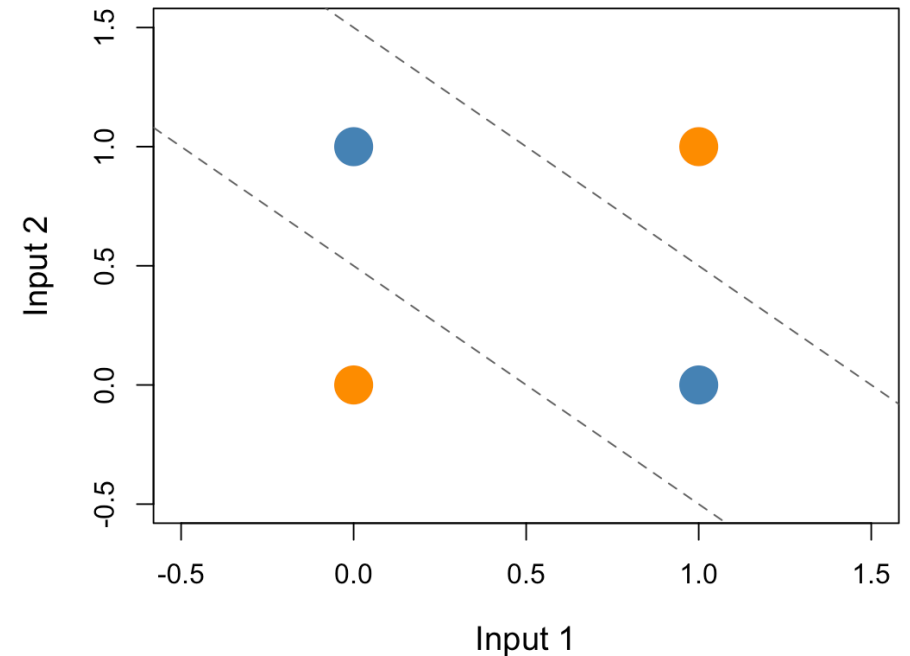
1. Explain what a hidden layer adds to a single-unit network, and why XOR needs one
2. Build a two-layer network (one hidden layer, one output unit) and run its forward pass
3. Explain backpropagation as the delta rule applied twice: once at the output, once “carried backward” into the hidden layer
4. Train a hidden-layer network from scratch and watch it solve XOR
5. Recognize hidden units as still just perceptrons stacked onto each other (i.e., deeper networks)

# Quick recap: the wall we hit

XOR Data

| Input 1 | Input 2 | Target |
|---------|---------|--------|
| 0       | 0       | 0      |
| 0       | 1       | 1      |
| 1       | 0       | 1      |
| 1       | 1       | 0      |

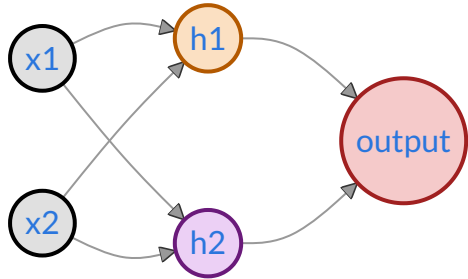
The two-line boundary



- Previously: The XOR problem. Can a single perceptron separate blue from orange? In the tutorial, you tried training one on exactly this data, and watched the loss plateau well above zero, no matter how long you trained or where you started from
- That's not bad luck: it's a provable limitation. No single straight line separates these four points, so no single perceptron ever can either, however its weights are set
- The two dashed lines above are the intuition for today's fix: they box the blue points into a band, with both orange points left outside on either side. Again, no *single* line can do that, but two lines *together* can. Two lines means two units

# Building a network: the hidden layer

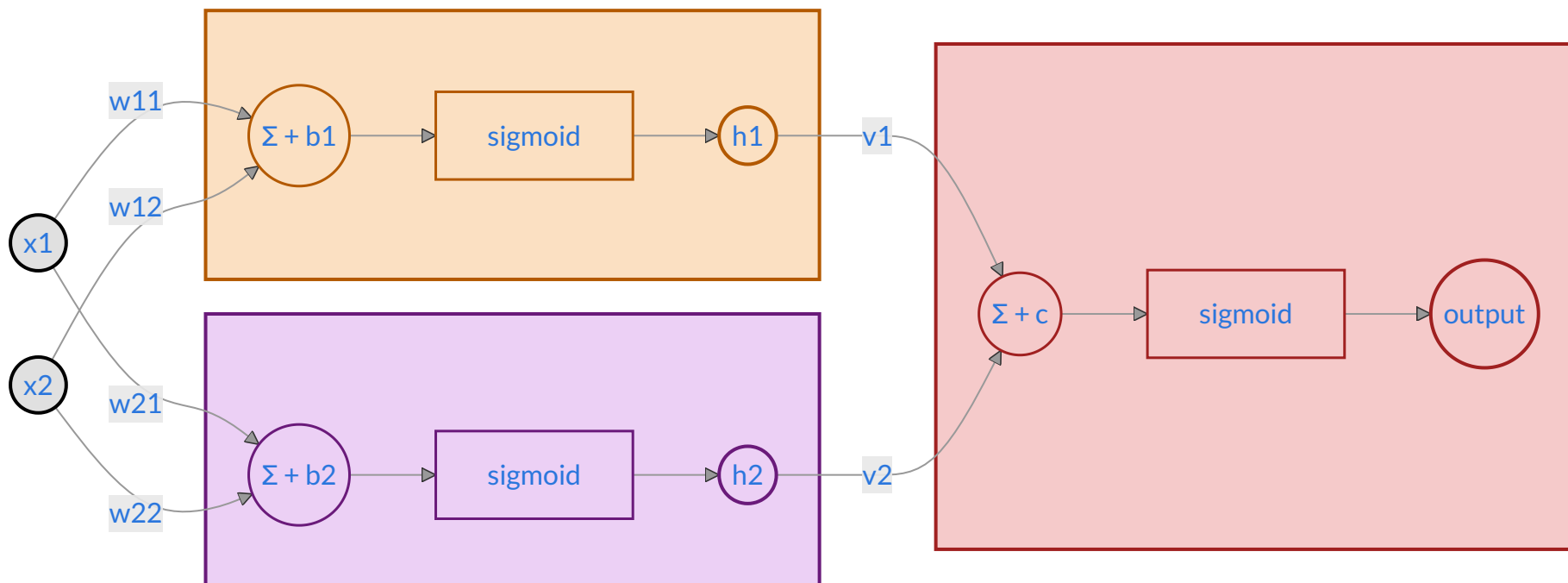
# From one unit to many: the hidden layer



■ Perceptron 1 ( $\rightarrow h_1$ ) ■ Perceptron 2 ( $\rightarrow h_2$ ) ■ Output perceptron

- Instead of one unit mapping inputs straight to an output, add a **hidden layer**: several units in between, each one an ordinary perceptron, each drawing its own line
- A further **output unit** takes those hidden units' outputs *as its inputs*, and combines them
- Nothing here is a new kind of unit. Every *hidden* and *output* circle in this diagram is the exact same sigmoid perceptron from previously. What's new is *arranging several of them in layers*

# Zooming in: each hidden and output unit is a perceptron from previously



■ Perceptron 1 ( $\rightarrow h_1$ ) ■ Perceptron 2 ( $\rightarrow h_2$ ) ■ Output perceptron

- This is the exact same “weighted sum, then sigmoid” picture from previously, drawn three times and wired together. The top chain ( $\Sigma + b_1 \rightarrow \text{sigmoid}$ ) is the first hidden perceptron; its output (the number it hands onward) is what we call  $h_1$ . The middle chain is a *second, independent* perceptron with the same two inputs but its own weights, and its output is  $h_2$ . The output unit is a *third* perceptron, except its two inputs happen to be  $h_1$  and  $h_2$  instead of  $x_1$  and  $x_2$
- The simpler diagram on the previous slide is exactly this picture with the internals folded away, which is a useful shorthand once you already know what’s inside each circle

# A concrete architecture for more complex problems, like the XOR (1/2)

- 2 inputs, 2 hidden units, 1 output unit
- Each hidden unit has its own weights and bias, exactly like previously:

$$h_1 = \sigma(w_{11}x_1 + w_{12}x_2 + b_1) \quad h_2 = \sigma(w_{21}x_1 + w_{22}x_2 + b_2)$$

- The output unit takes  $h_1$  and  $h_2$  as its two inputs:

$$\text{output} = \sigma(v_1 h_1 + v_2 h_2 + c)$$

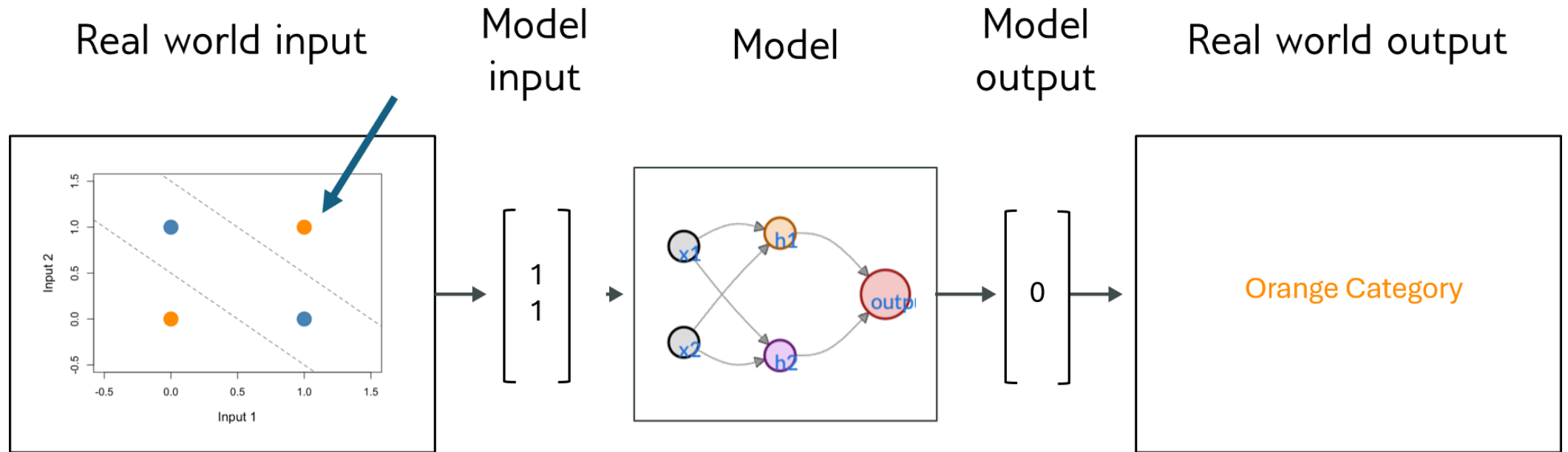
- Here,  $\sigma$  is the same sigmoid function from previously:  $\sigma(x) = \frac{1}{1 + e^{-x}}$  ; The Greek letter is just shorthand so the equations don't get cluttered with `sigmoid(...)` written out several times

# A concrete architecture for more complex problems, like the XOR (2/2)

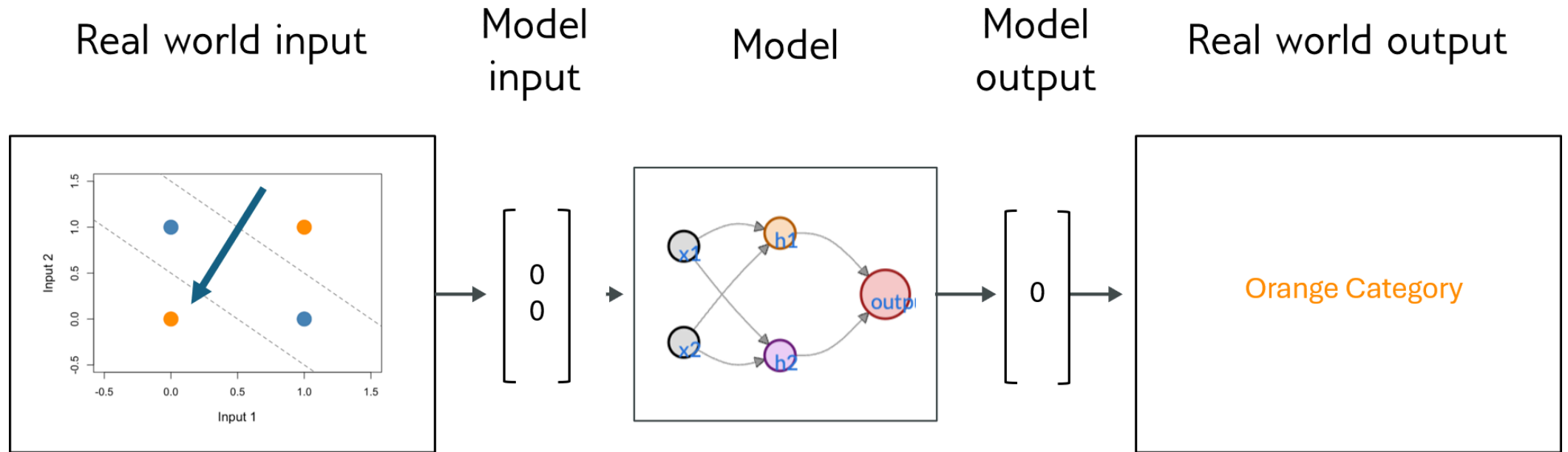
- Recall the tutorial generalized the perceptron to take a weight vector of any length. A hidden layer is the natural next step: a weight *matrix*  $W1$  (one row per hidden unit), plus a weight vector  $w2$  for the output layer

```
1 sigmoid <- function(x) {  
2   1 / (1 + exp(-x))  
3 }  
4  
5 # forward(): runs one input vector through the whole network and returns both  
6 # the hidden layer's activations (h) and the final output.  
7 forward <- function(x, W1, b1, w2, b2) {  
8   h <- sigmoid(W1 %*% x + b1)          # all hidden units' outputs, computed at once  
9   output <- sigmoid(sum(w2 * h) + b2) # output unit takes h as its input  
10  list(h = as.vector(h), output = output)  
11 }
```

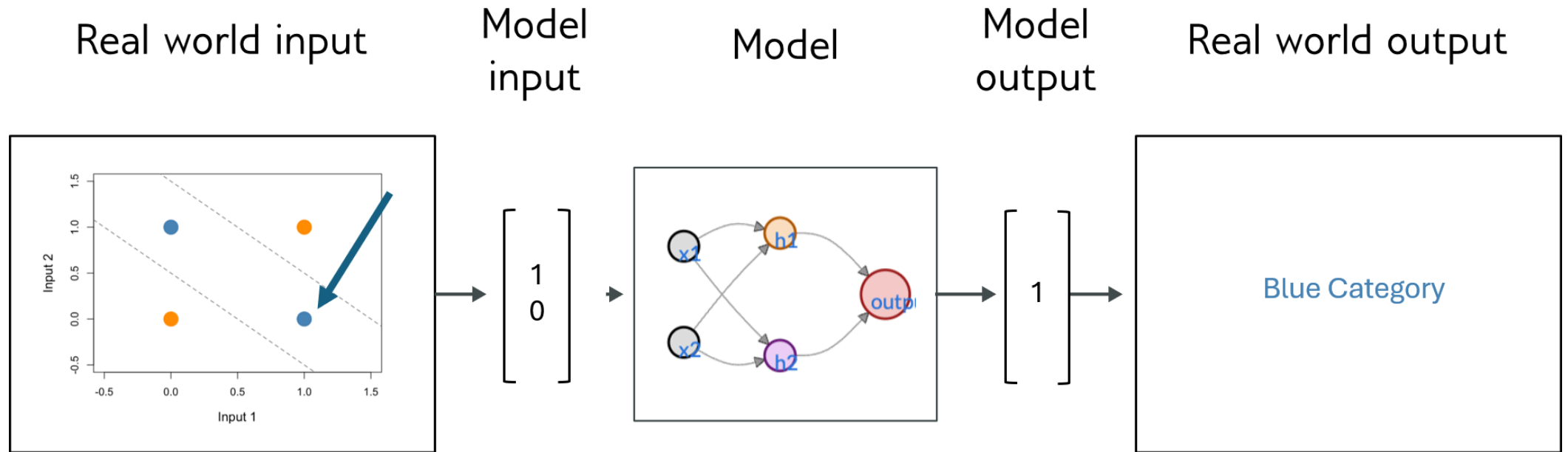
# the XOR pipeline (1/4)



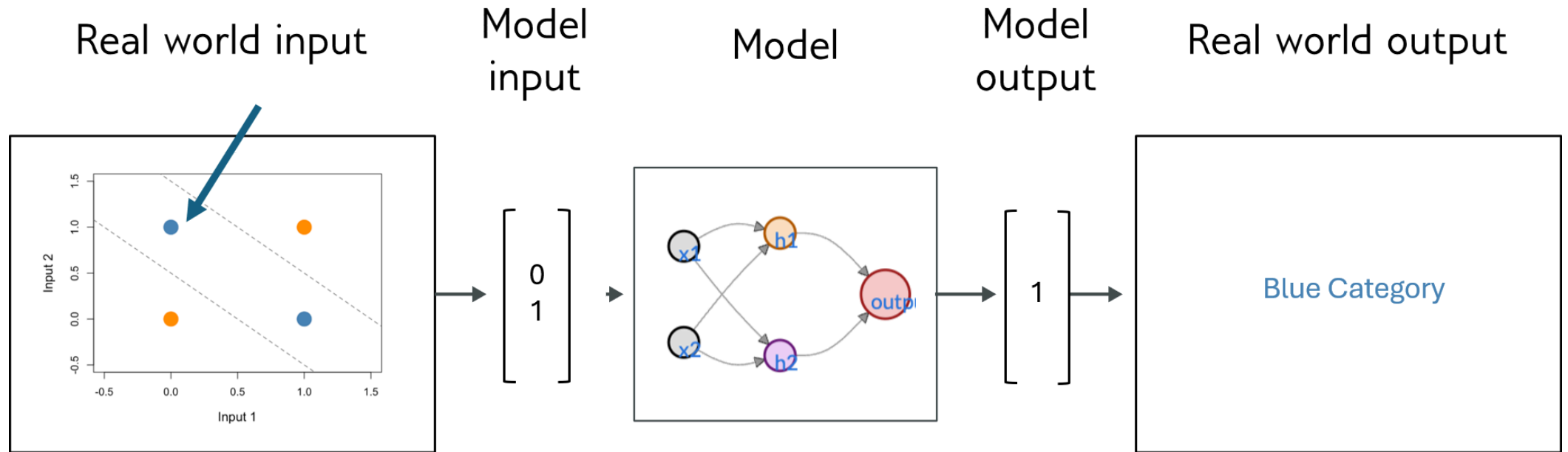
# the XOR pipeline (2/4)



# the XOR pipeline (3/4)



# the XOR pipeline (4/4)



**Proof by hand: hidden  
layers can solve XOR**

# What is **OR**? What is **NAND**?

| $x_1$ | $x_2$ | <b>OR</b> | <b>NAND</b> | XOR | category      |
|-------|-------|-----------|-------------|-----|---------------|
| 0     | 0     | 0         | 1           | ?   | 0<br>(orange) |
| 0     | 1     | 1         | 1           | ?   | 1 (blue)      |
| 1     | 0     | 1         | 1           | ?   | 1 (blue)      |
| 1     | 1     | 1         | 0           | ?   | 0<br>(orange) |

- **OR** is 1 whenever *at least one* input is 1, it fails (is 0) only at (0, 0)
- **NAND** (“not AND”) is 1 in every case *except* when *both* inputs are 1, it fails only at (1, 1)
- Each one “vetoes” exactly one corner of the table, and it’s a *different* corner for each
- **Fill in the XOR column yourselves:** The XOR operation outputs 1 if and only if its two binary inputs are different
- Now check XOR against the category column. XOR is 1 exactly where **OR** and **NAND** are *both* 1, and 0 wherever either one fails

# Proof it can be done: hand-picked weights (1/2)

```
1 # Row 1: x1=0, x2=0 -- OR should fail (output near 0)
2 x1 <- 0; x2 <- 0
3 sigmoid(20*x1 + 20*x2 - 10) # hidden unit 1 (OR): near 0
```

[1] 4.539787e-05

```
1 x1 <- 0; x2 <- 0
2 sigmoid(-20*x1 - 20*x2 + 30) # hidden unit 2 (NAND): near 1
```

[1] 1

```
1 # Row 2: x1=0, x2=1 -- OR holds
2 x1 <- 0; x2 <- 1
3 sigmoid(20*x1 + 20*x2 - 10) # near 1
```

[1] 0.9999546

```
1 x1 <- 0; x2 <- 1
2 sigmoid(-20*x1 - 20*x2 + 30) # near 1
```

[1] 0.9999546

```
1 # Row 3: x1=1, x2=0 -- OR holds
2 x1 <- 1; x2 <- 0
3 sigmoid(20*x1 + 20*x2 - 10) # near 1
```

[1] 0.9999546

```
1 x1 <- 1; x2 <- 0
2 sigmoid(-20*x1 - 20*x2 + 30) # near 1
```

[1] 0.9999546

```
1 # Row 4: x1=1, x2=1 -- OR holds
2 x1 <- 1; x2 <- 1
3 sigmoid(20*x1 + 20*x2 - 10) # near 1
```

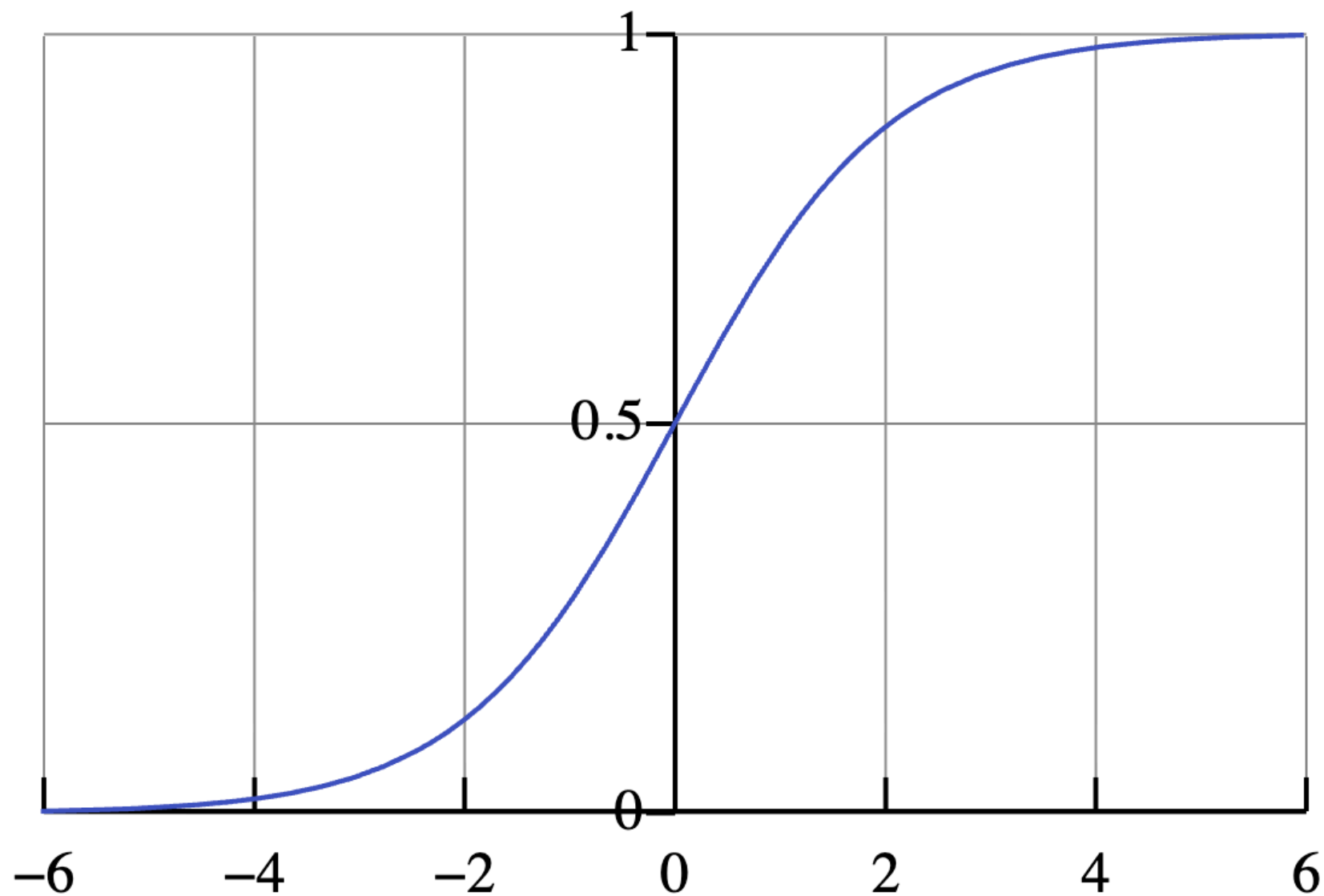
[1] 1

```
1 x1 <- 1; x2 <- 1
2 sigmoid(-20*x1 - 20*x2 + 30) # NAND should fail: near 0
```

[1] 4.539787e-05

That pattern across all four rows — **OR**: (0,1,1,1), **NAND**: (1,1,1,0) — is exactly each gate's truth table.

# Proof it can be done: hand-picked weights (1/2)



# Proof it can be done: hand-picked weights (2/2)

Here's the same check again, all four rows at once via `forward()`, now that you've seen each one on its own:

```
1 W1 <- matrix(c(20, 20, -20, -20), nrow = 2, byrow = TRUE) # row 1 = hidden unit 1's (x1, x2) weights
2                                     # row 2 = hidden unit 2's (x1, x2) weights
3 b1 <- c(-10, 30) # hidden unit 1's bias, hidden unit 2's bias
4 w2 <- c(20, 20) # output unit's weights on (h1, h2)
5 b2 <- -30 # output unit's bias
6
7 for (x1 in c(0, 1)) {
8   for (x2 in c(0, 1)) {
9     result <- forward(c(x1, x2), W1, b1, w2, b2)
10    cat(sprintf("x = (%d, %d) -> h = (%.3f, %.3f) -> output = %.3f (target = %d)\n",
11              x1, x2, result$h[1], result$h[2], result$output, xor(x1, x2)))
12  }
13 }
```

```
x = (0, 0) -> h = (0.000, 1.000) -> output = 0.000 (target = 0)
x = (0, 1) -> h = (1.000, 1.000) -> output = 1.000 (target = 1)
x = (1, 0) -> h = (1.000, 1.000) -> output = 1.000 (target = 1)
x = (1, 1) -> h = (1.000, 0.000) -> output = 0.000 (target = 0)
```

- Hidden unit 1 behaves like **OR**; hidden unit 2 behaves like **NAND**; the output combines them as **AND** — and OR-AND-NOT-AND is exactly XOR
- The large weights ( $\pm 20$ ) push each sigmoid close to a hard 0/1 step, which is why the outputs land so close to the true targets
- This proves a two-hidden-unit network *can* represent XOR. In a few slides we'll look at how would training ever *find* weights like these on its own (using a method known as backpropagation).

# Each hidden unit's own boundary line(1/2)

Each hidden unit is a perceptron, so each has the same straight-line boundary formula from previous lectures:  $w_1 x_1 + w_2 x_2 + b = 0$ .

Hidden unit 1: **OR**

$$20x_1 + 20x_2 - 10 = 0$$

↓

$$x_1 + x_2 = 0.5$$

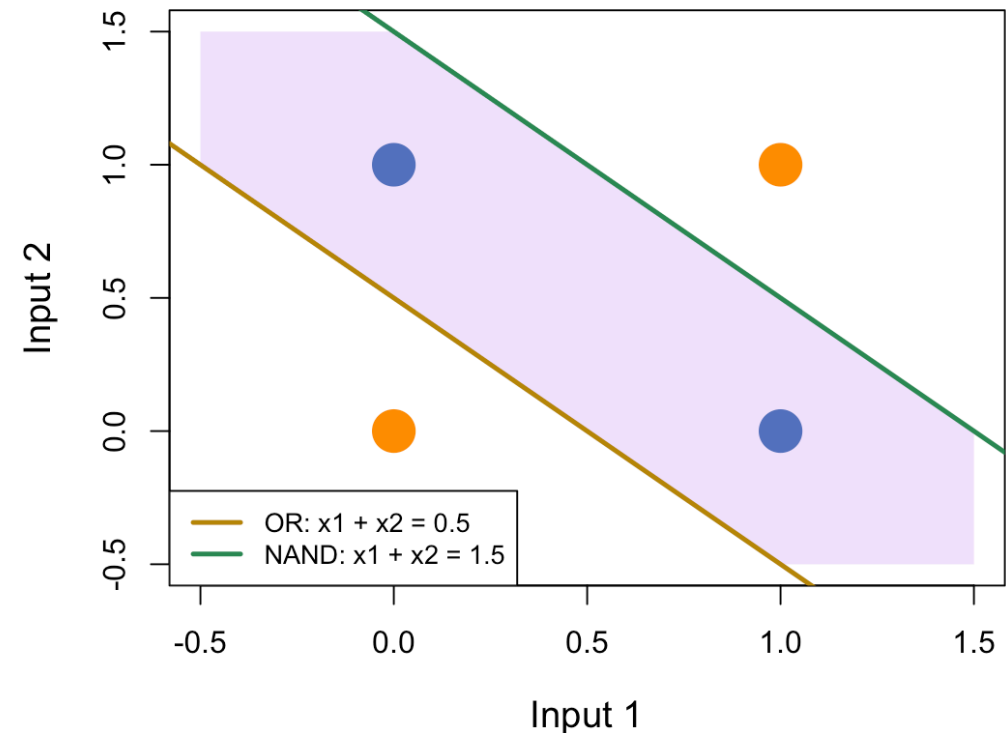
Hidden unit 2: **NAND**

$$-20x_1 - 20x_2 + 30 = 0$$

↓

$$x_1 + x_2 = 1.5$$

Both hidden units' boundaries



# Each hidden unit's own boundary line(2/2)

- Both lines have the *same slope* (-1) which is not a coincidence: hidden unit 1's weights are (20,20), hidden unit 2's are (-20,-20), and both ratios give slope -1. Same slope means the lines are **parallel** and never meet, at any distance. "0.5" and "1.5" are each line's own intercept, not the coordinates of some meeting point
- What the shaded band means, precisely: for any  $(x_1, x_2)$  point *inside* it, both hidden units' **output values** (not the network's final *output*) are close to 1. That is,  $h_1(x_1, x_2) \approx 1$  and  $h_2(x_1, x_2) \approx 1$  simultaneously, at that point. It's a statement about the two numbers each hidden unit outputs, nothing about relocating any point anywhere

Concrete example:  $(x_1, x_2) = (0, 1)$

|                         | Weighted sum               | Sigmoid output               |
|-------------------------|----------------------------|------------------------------|
| Hidden unit 1<br>(OR)   | $20(0) + 20(1) - 10 = 10$  | $\sigma(10) \approx 0.99995$ |
| Hidden unit 2<br>(NAND) | $-20(0) - 20(1) + 30 = 10$ | $\sigma(10) \approx 0.99995$ |

So, for this input, **both hidden units are essentially "on."**

- That's a *different* fact from something coming up on the very next slide: when  $h_1$  and  $h_2$  get used as coordinates in their own right (which is a whole new 2D space) the two blue points end up landing at the exact same location in that space. That's a separate space entirely ( $h_1, h_2$ -space, not  $x_1, x_2$ -space)

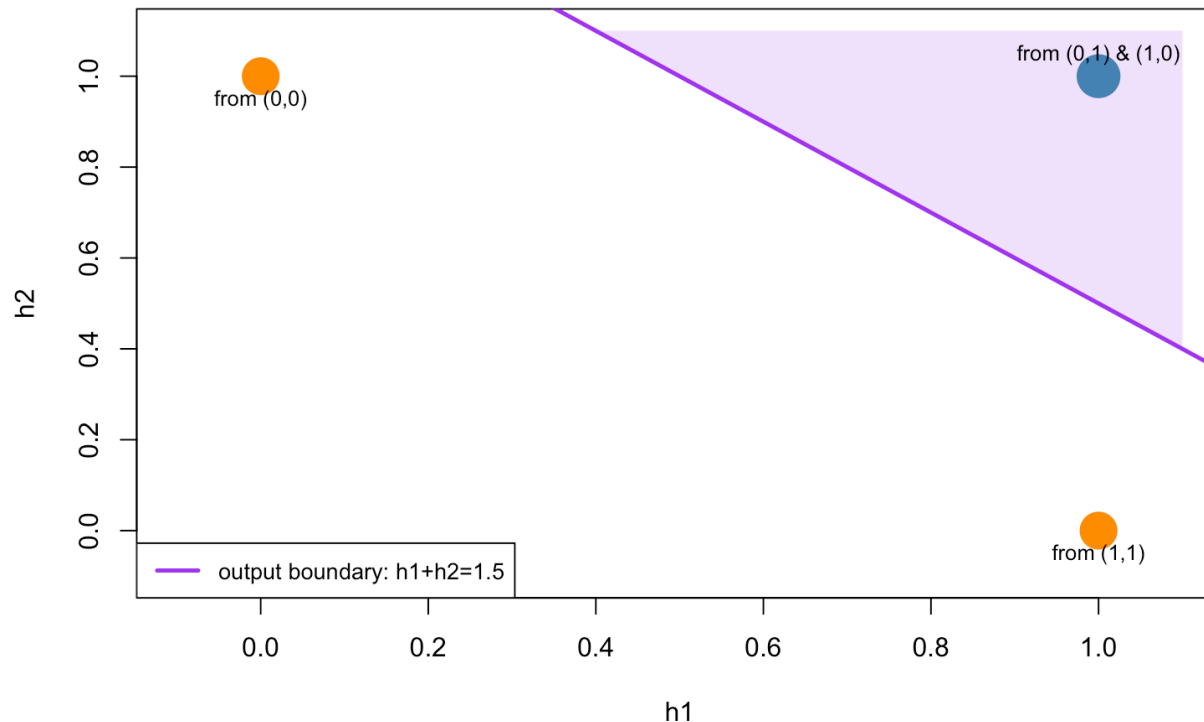
# Why does the output unit's job turn out to be easy? (1/2)

The output unit is *also* just a perceptron, whose weight sum is also one straight line. So how does one straight line correctly separate points that weren't linearly separable to begin with? By the time the output unit sees the data, it isn't looking at  $(x_1, x_2)$  anymore rather it's looking at  $(h_1, h_2)$ :

```
1 for (x1 in c(0, 1)) {  
2   for (x2 in c(0, 1)) {  
3     result <- forward(c(x1, x2), W1, b1, w2, b2)  
4     h <- result$h  
5     cat(sprintf("(x1, x2) = (%d, %d)  ->  (h1, h2) = (%.0f, %.0f)\n", x1, x2, h[1], h[2]))  
6   }  
7 }
```

|                       |               |                       |
|-----------------------|---------------|-----------------------|
| $(x_1, x_2) = (0, 0)$ | $\rightarrow$ | $(h_1, h_2) = (0, 1)$ |
| $(x_1, x_2) = (0, 1)$ | $\rightarrow$ | $(h_1, h_2) = (1, 1)$ |
| $(x_1, x_2) = (1, 0)$ | $\rightarrow$ | $(h_1, h_2) = (1, 1)$ |
| $(x_1, x_2) = (1, 1)$ | $\rightarrow$ | $(h_1, h_2) = (1, 0)$ |

The output unit's real decision boundary



# Why does the output unit's job turn out to be easy? (2/2)

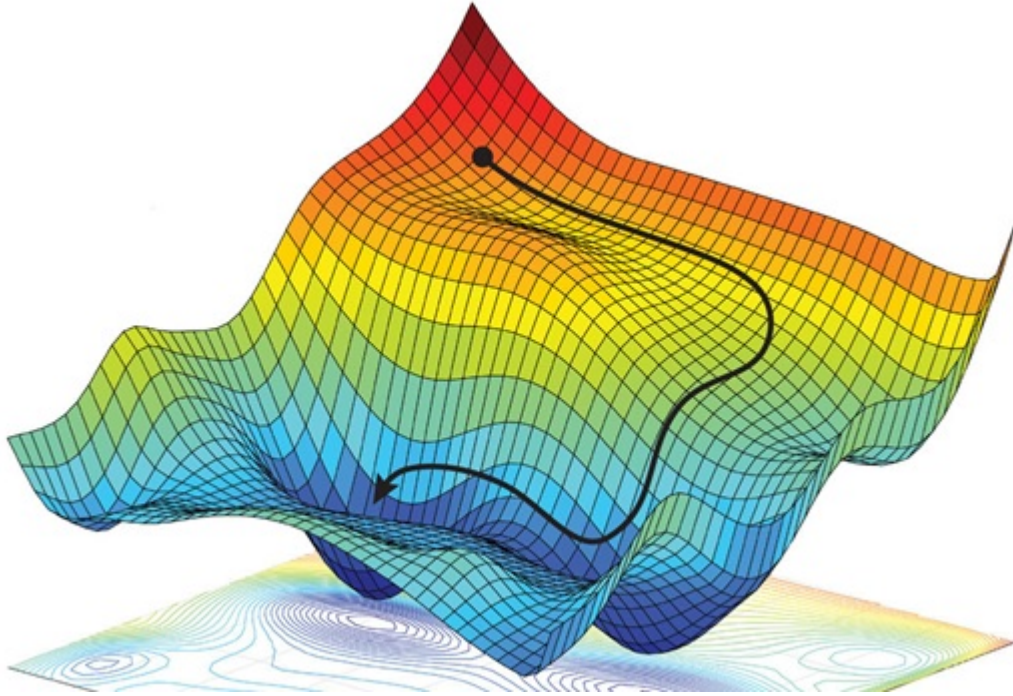
- In  $(x_1, x_2)$  space, the four XOR points aren't linearly separable and that's the whole problem from previously
- But look what happened in  $(h_1, h_2)$  space: the two points that *should* share a category (when  $x_1$  and  $x_2$  differ in their values, category 1) both landed at the exact same spot,  $(1, 1)$ . The two points that should be in the *other* category landed at two different spots,  $(0, 1)$  and  $(1, 0)$
- The output's boundary,  $h_1 + h_2 = 1.5$ , is a genuine, single straight line in *this* space separating one point from two others
- This is the real trick of a hidden layer: it doesn't avoid linear separation, it **reshapes the space** until linear separation becomes possible. The output unit's job really is just "find one line" in its own space and the hard work already happened one layer earlier

**Where do these weights  
come from?**

**Backpropagation**

# Where do these weights come from?

## Going down the error landscape



# Where do these weights come from?

## Backpropagation

- The delta rule previously updated a single unit's weights using its own error signal
- A hidden unit has a problem: it doesn't have a target. The dataset tells us what the *output* should have been but nobody tells hidden unit 1 what *it* should have output
- Backpropagation's answer: **send the output's error backward** through the network, so each hidden unit gets a share of the blame proportional to how much it contributed to the output's mistake
- Mechanically, it's the delta rule, applied twice: once at the output layer (same formula as previously), once more at the hidden layer (new formula)

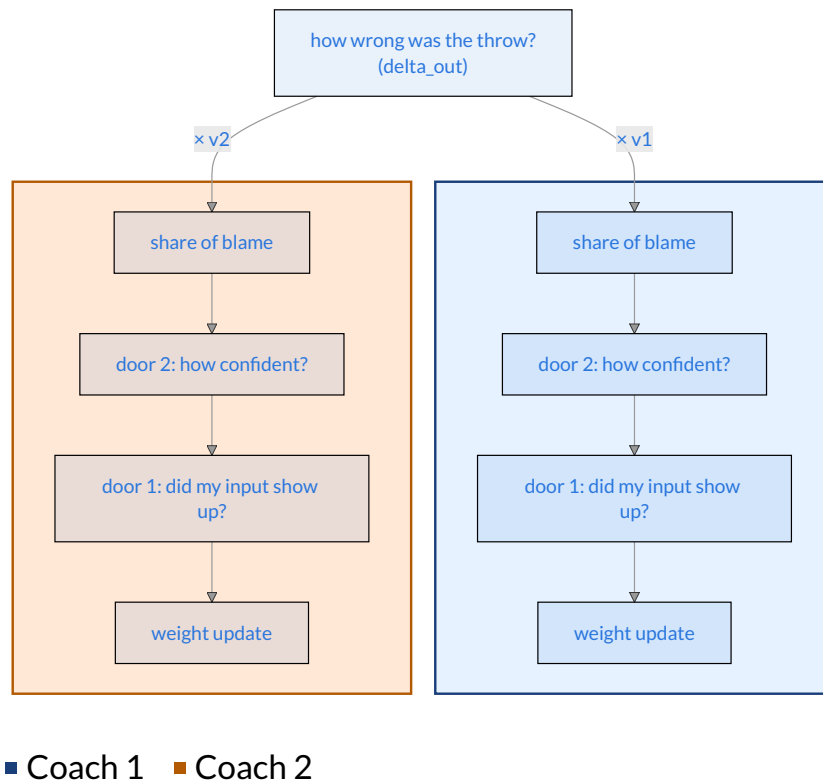
# Before the formulas: a coaching story.



# Before the formulas: a coaching story.

- Last week: one athlete, throwing darts, adjusting their own grip after every throw
- Today: the athlete doesn't throw alone anymore. Before each throw, two assistant coaches each whisper their own advice and the athlete blends both suggestions into a single throw
- The throw still lands somewhere, still misses by some amount but now, whose fault was that?
- Here's the key idea: when the head coach hands out blame after a bad throw, it doesn't go to one person. It gets split between the two assistants **in proportion to how much attention the athlete actually paid to each one's advice**. The assistant the athlete mostly ignored gets barely any blame; the one the athlete leaned on heavily gets a much bigger share
- "How wrong was the whole throw" is already settled at this point as that question only gets asked once. What each assistant asks next is different: given my share of the blame, was I already confident in my own advice, and did my own input even factor into what I said this time? Same two questions as last week, now asked separately by each coach, about their own advice

# How much of the blame is mine?



- Blame flows **backward**, from the throw's outcome, back through each assistant's own connection strength to the athlete
- Each assistant's *share* of the blame depends on two things multiplied together: how wrong the final throw was (already known from the start, top of the left figure), and how much the athlete actually listened to *that* assistant ( $v_i$ )
- Then, exactly like last week, each assistant asks their own door 2: given my share of the blame, how confident was I in what I said? An assistant who was already very sure of their advice gets a smaller correction than one who was genuinely guessing, for exactly the same reason as last week
- **And still exactly like last week**, each assistant then asks their own door 1: did *my* own input even show up this time? Nothing about this step has changed from a single unit. It's the same question, asked once per weight, using whichever input actually feeds that weight

# The two-step update rule

# Reminder: last week's delta rule

Before the two-step version, here's the one-step version you already know, unchanged:

$$\Delta w = \eta \cdot (\text{target} - \text{predicted}) \cdot \text{predicted} \cdot (1 - \text{predicted}) \cdot \text{input}$$

- How wrong was I?:  $(\text{target} - \text{predicted})$
- Door 2: how confident was I already?:  $\text{predicted} \cdot (1 - \text{predicted})$
- Door 1: did this weight's own input show up this time?:  $\text{input}$
- This is exactly **Step 1** on the next slide, applied to the output unit which is the same formula, same three questions. **Step 2** is the new idea for today: what does a *hidden* unit's version of “how wrong was I” even mean, when nobody hands it a target directly?

# The two-step update (1/3) : naming the symbols

Step 1 — output layer (exactly the delta rule from previously):

$$\delta_{\text{out}} = (\text{target} - \text{output}) \cdot \text{output} \cdot (1 - \text{output})$$

$$\Delta v_i = \eta \cdot \delta_{\text{out}} \cdot h_i \quad \Delta c = \eta \cdot \delta_{\text{out}}$$

Step 2 — hidden layer (the new part):

$$\delta_{h_i} = \delta_{\text{out}} \cdot v_i \cdot h_i \cdot (1 - h_i)$$

$$\Delta w_{ij} = \eta \cdot \delta_{h_i} \cdot x_j \quad \Delta b_i = \eta \cdot \delta_{h_i}$$

- What each symbol is:  $v_i$  is the output unit's weight *on* hidden unit  $i$ , and  $c$  is the output unit's bias (both from the architecture slide);  $w_{ij}$  is hidden unit  $i$ 's weight on input  $j$ , and  $b_i$  is hidden unit  $i$ 's own bias
- We call the network's final answer **output** rather than **predicted** (as previously) purely to keep it visually distinct from each hidden unit's own  $h_i$ , now that there's more than one unit's output to keep straight

# The two-step update (2/3) : connection to analogy

Step 1 — output layer (exactly the delta rule from previously):

$$\delta_{\text{out}} = (\text{target} - \text{output}) \cdot \text{output} \cdot (1 - \text{output})$$

$$\Delta v_i = \eta \cdot \delta_{\text{out}} \cdot h_i \quad \Delta c = \eta \cdot \delta_{\text{out}}$$

Step 2 — hidden layer (the new part):

$$\delta_{h_i} = \delta_{\text{out}} \cdot v_i \cdot h_i \cdot (1 - h_i)$$

$$\Delta w_{ij} = \eta \cdot \delta_{h_i} \cdot x_j \quad \Delta b_i = \eta \cdot \delta_{h_i}$$

- In the coaching story,  $v_i$  is exactly “how much attention was paid to coach  $i$ ’s advice”. A hidden unit with a large  $v_i$  gets a large share of  $\delta_{\text{out}}$ ’s blame; one with  $v_i$  near 0 gets almost none, however wrong the final throw was
- $h_i \cdot (1 - h_i)$  inside  $\delta_{h_i}$  is that hidden unit’s own door 2, asked *after* its share of the blame arrives which is the same question, same formula, from Neural Networks 1 single unit
- $h_i$  is hidden unit  $i$ ’s own output (its own sigmoid activation) exactly analogous to **output** above, just for one hidden unit instead of the whole network.

# The two-step update (3/3) : how big a correction, exactly?

Step 1 — output layer (exactly the delta rule from previously):

$$\delta_{\text{out}} = (\text{target} - \text{output}) \cdot \text{output} \cdot (1 - \text{output})$$

$$\Delta v_i = \eta \cdot \delta_{\text{out}} \cdot h_i \quad \Delta c = \eta \cdot \delta_{\text{out}}$$

Step 2 — hidden layer (the new part):

$$\delta_{h_i} = \delta_{\text{out}} \cdot v_i \cdot h_i \cdot (1 - h_i)$$

$$\Delta w_{ij} = \eta \cdot \delta_{h_i} \cdot x_j \quad \Delta b_i = \eta \cdot \delta_{h_i}$$

- Compare the two formulas in step 1 and step 2. In step 2, `delta_out` plays the same role as `(target - output)` used in step 1
- Intuition for  $\delta_{\text{out}}$ : large in magnitude means the output was very wrong (a big correction is needed); close to zero means the output was already close to target (barely nudge anything). Two concrete examples, both with target = 1:

| output | how wrong    | $\delta_{\text{out}}$ |
|--------|--------------|-----------------------|
| 0.10   | way off      | 0.0810                |
| 0.95   | nearly right | 0.0024                |

# A worked example (1/3)

One trial:  $x = (1, 0)$ , target = 1.

- Why “deliberately small” weights here, unlike the  $\pm 20$  hand-picked ones? XOR’s inputs are only ever 0 or 1 so with weights around magnitude 1, the weighted sum  $w_1 x_1 + w_2 x_2 + b$  stays roughly in the range -2 to 2, which keeps the sigmoid in its *responsive* middle zone (nothing rounds all the way to exactly 0 or 1). That’s the opposite goal from the hand-picked weights, which deliberately pushed the sigmoid toward a hard step. Small, non-saturated weights are what a network typically starts with (see `rnorm(..., sd = 1)` a few slides from now) and this example is meant to look like a realistic mid-training snapshot, not another hand-engineered solution

```
1 W1 <- matrix(c(1, -1, 0.5, 0.5), nrow = 2, byrow = TRUE)
2 b1 <- c(0, 0)
3 w2 <- c(1, -1)
4 b2 <- 0
5 lr <- 0.5
6
7 x <- c(1, 0); target <- 1
8 fwd <- forward(x, W1, b1, w2, b2) # forward pass: get this trial's h and output
9 h <- fwd$h; output <- fwd$output
10
11 delta_out <- (target - output) * output * (1 - output) # step 1: output layer's delta
12 delta_hidden <- delta_out * w2 * h * (1 - h) # step 2: each hidden unit's delta
13
14 round(c(h1 = h[1], h2 = h[2], output = output, delta_out = delta_out), 4)
```

| h1     | h2     | output | delta_out |
|--------|--------|--------|-----------|
| 0.7311 | 0.6225 | 0.5271 | 0.1179    |

```
1 round(c(delta_h1 = delta_hidden[1], delta_h2 = delta_hidden[2]), 4)
```

| delta_h1 | delta_h2 |
|----------|----------|
| 0.0232   | -0.0277  |

# A worked example (2/3)

```
1 # Each hidden unit's row of W1 updates the same way as a single unit always has:
2 # weight <- weight + lr * delta * input. We just do it once per hidden unit, by
3 # hand, so every entry is visible on the page rather than hidden inside a function.
4 W1_new <- W1
5 W1_new[1, 1] <- W1[1, 1] + lr * delta_hidden[1] * x[1] # hidden unit 1, weight on x1
6 W1_new[1, 2] <- W1[1, 2] + lr * delta_hidden[1] * x[2] # hidden unit 1, weight on x2
7 W1_new[2, 1] <- W1[2, 1] + lr * delta_hidden[2] * x[1] # hidden unit 2, weight on x1
8 W1_new[2, 2] <- W1[2, 2] + lr * delta_hidden[2] * x[2] # hidden unit 2, weight on x2
9
10 # R has a one-line shortcut for exactly this pattern: outer(a, b) builds a matrix
11 # where entry [i,j] = a[i] * b[j] -- precisely the four lines above, done at once.
12 # The equivalent one-liner for the update above is:
13 # W1_new <- W1 + lr * outer(delta_hidden, x)
14 # We'll switch to this shorthand from here on, now that you've seen what it expands to.
15
16 b1_new <- b1 + lr * delta_hidden          # one bias update per hidden unit
17 w2_new <- w2 + lr * delta_out * h         # output weights, same delta rule as previously
18 b2_new <- b2 + lr * delta_out            # output bias
19
20 round(W1_new, 4); round(b1_new, 4); round(w2_new, 4); round(b2_new, 4)
```

```
      [,1] [,2]
[1,] 1.0116 -1.0
[2,] 0.4861  0.5

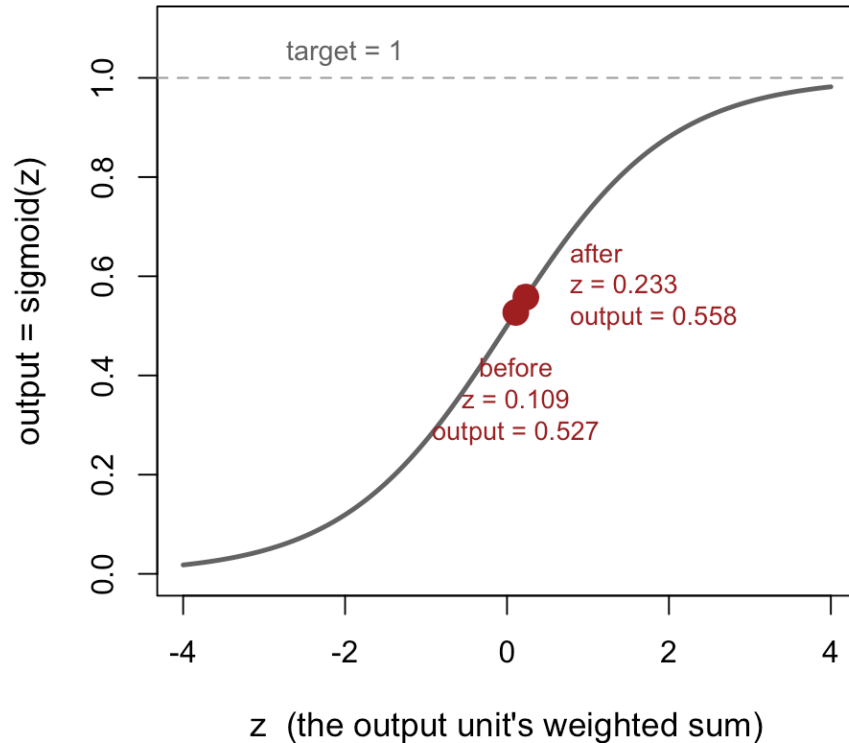
[1]  0.0116 -0.0139

[1]  1.0431 -0.9633

[1] 0.0589
```

- `delta_hidden` comes out **0.0232** for hidden unit 1, but **-0.0277** for hidden unit 2. These are opposite signs, even though both are being blamed for the *same* output error. Why:  $\delta_{h_i} = \delta_{\text{out}} \cdot v_i \cdot h_i(1 - h_i)$ . The  $h_i(1 - h_i)$  term can't be negative (a probability times one-minus-itself), and  $\delta_{\text{out}}$  is positive here too (the output was too low). So the *only* thing that can flip the sign is  $v_i$ , each hidden unit's own connection weight to the output

# A worked example (3/3): how backprop moves the output closer to the target



- The output unit just adds things up:  $z = v_1 h_1 + v_2 h_2 + c$ , and then squashes  $z$  with the sigmoid. Right now  $z = (1)(0.731) + (-1)(0.622) + 0 = 0.109$ , which gives an output of 0.527. The target is 1, and the sigmoid only approaches 1 as  $z$  grows, so we need  $z$  to go **up**
- Each term in that sum can grow in two ways: by changing the weight  $v_i$  (step 1), or by changing the hidden activation  $h_i$  (step 2)
- **Step 1 changes the  $v_i$ .** Both go *up* ( $1 \rightarrow 1.0431$  and  $-1 \rightarrow -0.9633$ ), because the output was too low and both hidden units were active. So  $v_1 h_1$  gets bigger and  $v_2 h_2$  gets less negative
- **Step 2 changes the  $h_i$ ,** by changing the hidden units' own weights. Raising  $h_1$  helps, because  $v_1$  is positive. But raising  $h_2$  would make  $v_2 h_2$  more negative and pull  $z$  down, so  $h_2$  is pushed *down*. You can see this in the updated hidden weights on the previous slide: hidden unit 1's weight on  $x_1$  rises ( $1 \rightarrow 1.0116$ ) while hidden unit 2's falls ( $0.5 \rightarrow 0.4861$ ). The weights on  $x_2$  don't change, because  $x_2 = 0$  on this trial
- Together, one update moves  $z$  from 0.109 to 0.233 and the output from 0.527 to 0.558, which the next slide checks

# Did that update actually help?

Same trial, same target. Run the *new* weights back through `forward()` and check.

```
1 h_new_fwd <- forward(x, W1_new, b1_new, w2_new, b2_new)
2 output_new <- h_new_fwd$output
3
4 round(c(output_before = output, output_after = output_new, target = target), 4)
```

| output_before | output_after | target |
|---------------|--------------|--------|
| 0.5271        | 0.5580       | 1.0000 |

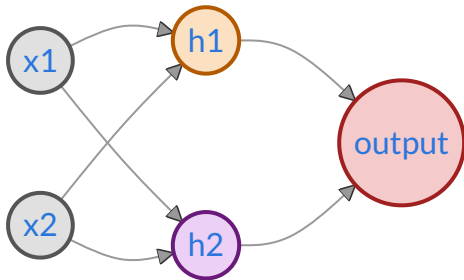
```
1 round(c(error_before = abs(target - output), error_after = abs(target - output_new)), 4)
```

| error_before | error_after |
|--------------|-------------|
| 0.4729       | 0.4420      |

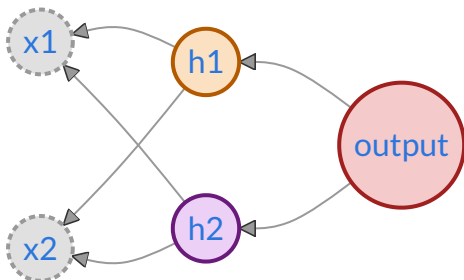
- `output` moves from 0.5271 to 0.5580 which is still far from the target of 1, but closer
- The error shrank from 0.4729 to 0.4420 which is only one trial, and one small step, but in the right direction, exactly like last week's single-unit check. Two layers and a backward-blame step don't change that basic promise: each update, however small, should move the network's answer toward the target it just got wrong

# Why “back”-propagation? (1/2)

**Forward pass:** inputs flow left to right, through the hidden layer, to a prediction. Every input connects to every hidden unit:



**Backward pass:** the error at the output flows right to left, back through the same connections:



— weight updated,  $\delta$  computed here

⊘ input node is not a parameter, so no  $\delta$  here

Backprop updates weights at both layers: first the output layer’s own weights ( $h1 \rightarrow \text{output}$ ,  $h2 \rightarrow \text{output}$ ), then (using that same output delta) the four weights from the inputs into the hidden layer. What stops at the hidden layer is only the delta itself: The inputs,  $x_1$  and  $x_2$ , aren’t parameters, so there’s no delta to compute *for* them, only for units that have their own incoming weights to receive blame. The input nodes are shown dashed/greyed out to mark that, rather than suggest their weights change.

# Why “back”-propagation? (2/2)

- The forward pass is `forward()`, the function from a few slides ago
- Concretely, two computations happen on the way back: first  $\delta_{\text{out}}$  gets computed at the output (step 1, from ‘*the two-step update*’ a few slides ago); then that *same*  $\delta_{\text{out}}$  value gets reused, once per hidden unit, to compute each unit’s own  $\delta_{h_i}$ , which is the first half of step 2, and literally the two arrows from output into h1 and h2 in the diagram above
- The second half of step 2, updating each  $w_{ij}$  and  $b_i$  using that  $\delta_{h_i}$  together with the input  $x_j$ , happens along the hidden  $\rightarrow$  input connections. Those arrows don’t carry a new  $\delta$  forward (nothing to compute *for*  $x_1$  or  $x_2$  as they are not free parameters)
- One forward pass, one backward pass, per training example which is same “one example at a time” rhythm as the delta rule from Neural Networks 1, just with an extra step

# One trial vs. one epoch

- Everything on the last few slides (i.e., computing  $\delta_{\text{out}}$ , then each  $\delta_{h_i}$ , then updating every weight) happens once per **training example** (one row of the dataset), exactly like the delta rule previously
- One **epoch** = doing that, once, for *every* row of the dataset, in sequence. XOR has 4 rows, so one epoch is 4 trials: 4 rounds of forward pass, backward pass, and weight update
- The loss you'll see plotted (RMSE) gets computed once *per epoch*, across all 4 rows together. The loss is thus a summary of overall progress
- Training runs many epochs (thousands, in the code coming up): the same 4 trials, in the same order, over and over, with weights nudged a tiny bit after every single trial

# Training the network end to end

# Training in R (1/2)

```
1 # train_xor(): runs the whole training procedure from a random starting point.
2 # Outer loop = epochs (repeat the whole dataset many times).
3 # Inner loop = one trial per row: forward pass, compute both deltas, update every weight.
4 # Returns the loss (RMSD) after every epoch, plus the final trained weights.
5 train_xor <- function(seed, n_epochs = 3000, lr = 0.5) {
6   set.seed(seed) # same seed -> same random start, every time this runs
7   W1 <- matrix(rnorm(4, sd = 1), nrow = 2) # random starting weights: hidden layer (2 units x 2 inputs)
8   b1 <- rnorm(2, sd = 1) # random starting hidden-layer biases
9   w2 <- rnorm(2, sd = 1) # random starting output-layer weights
10  b2 <- rnorm(1, sd = 1) # random starting output-layer bias
11
12  loss_history <- numeric(n_epochs) # will hold one RMSD value per epoch
13  for (epoch in 1:n_epochs) {
14    for (i in 1:nrow(xor_data)) { # one trial = one row of xor_data
15      x <- c(xor_data$x1[i], xor_data$x2[i])
16      target <- xor_data$category[i]
17
18      fwd <- forward(x, W1, b1, w2, b2) # forward pass: get h and output for this trial
19      h <- fwd$h; output <- fwd$output
20
21      delta_out <- (target - output) * output * (1 - output) # step 1: output layer's delta
22      delta_hidden <- delta_out * w2 * h * (1 - h) # step 2: each hidden unit's delta
23
24      w2 <- w2 + lr * delta_out * h # update output weights
25      b2 <- b2 + lr * delta_out # update output bias
26      # outer(delta_hidden, x) is the one-line version of the four-entry-by-hand
27      # update from the worked example a couple of slides back -- equivalent to:
28      # W1[1,1] <- W1[1,1] + lr*delta_hidden[1]*x[1]; W1[1,2] <- W1[1,2] + lr*delta_hidden[1]*x[2]
29      # W1[2,1] <- W1[2,1] + lr*delta_hidden[2]*x[1]; W1[2,2] <- W1[2,2] + lr*delta_hidden[2]*x[2]
30      W1 <- W1 + lr * outer(delta_hidden, x) # update hidden weights (all at once, via outer product)
31      b1 <- b1 + lr * delta_hidden # update hidden biases
32    }
33    # after all 4 trials this epoch, compute RMSD across the whole dataset for the loss curve
34    # 1. Pre-allocate a vector to store the 4 predictions
35    preds <- numeric(nrow(xor_data))
36    # 2. Loop through each row one by one
37    for (i in 1:nrow(xor_data)) {
38      x_i <- c(xor_data$x1[i], xor_data$x2[i])
39      fwd <- forward(x_i, W1, b1, w2, b2)
```

# Training in R (2/2)

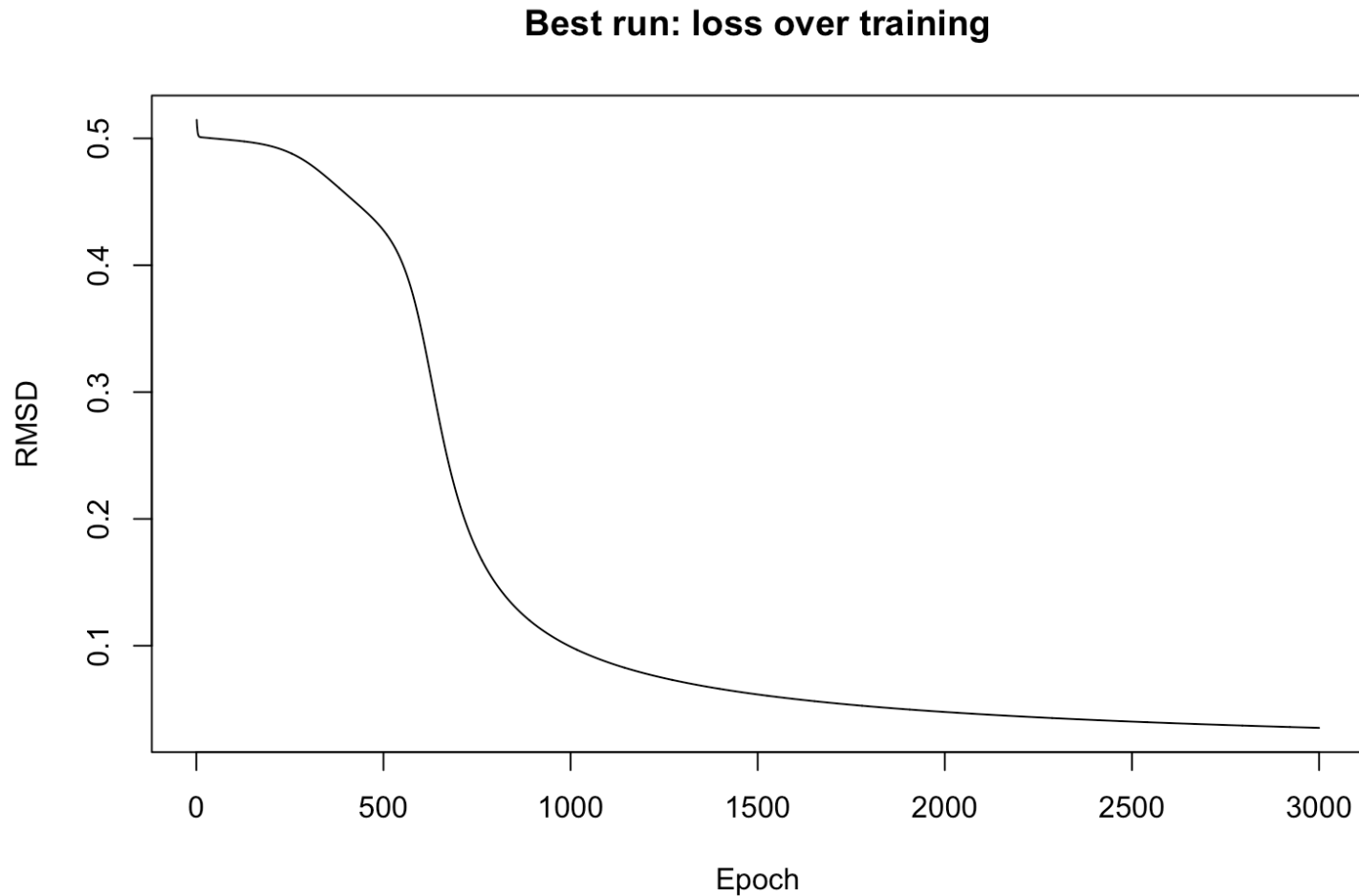
- A random starting point occasionally gets stuck (a real, known quirk of training small networks with plain gradient descent). The typical fix is to try a few random starts, and keep the best

```
1 # Try 5 independent random starts (seeds 1 to 5), and keep whichever one ends
2 # with the lowest final RMSD.
3 runs <- list()                                # will hold the 5 training runs
4 for (seed in 1:5) {
5   runs[[seed]] <- train_xor(seed)             # store the run that started from this seed
6 }
7 final_losses <- numeric(length(runs))          # one final RMSD per run
8 for (k in 1:length(runs)) {
9   history <- runs[[k]]$loss_history            # the loss after every epoch, for run k
10  final_losses[k] <- history[length(history)]   # keep only the last one
11 }
12 best_run <- runs[[which.min(final_losses)]]
13 round(final_losses, 4)
```

```
[1] 0.3565 0.0493 0.0354 0.0352 0.0599
```

# It works! (1/3)

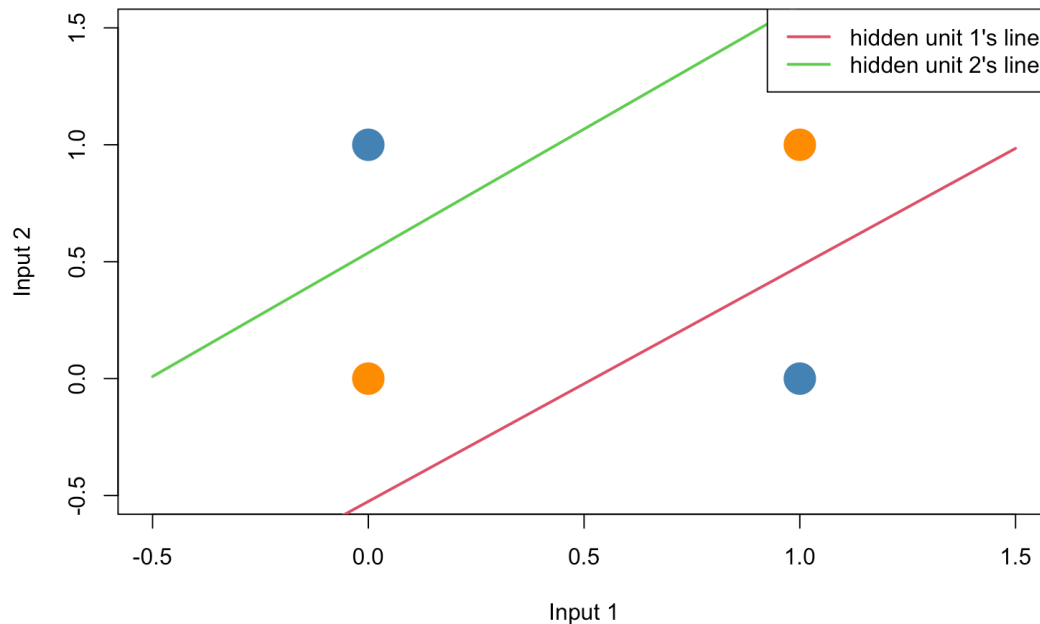
```
1 plot(best_run$loss_history, type = "l", xlab = "Epoch", ylab = "RMSD",  
2      main = "Best run: loss over training")
```



# It works! (2/3)

```
1 plot(xor_data$x1, xor_data$x2, pch = 19, cex = 3,  
2       col = ifelse(xor_data$category == 1, "steelblue", "darkorange"),  
3       xlab = "Input 1", ylab = "Input 2", xlim = c(-0.5, 1.5), ylim = c(-0.5, 1.5),  
4       main = "The two lines the network actually learned")  
5 x_seq <- seq(-0.5, 1.5, length.out = 100)  
6 for (i in 1:2) {  
7   # row i of W1 is hidden unit i's own weights/bias -> its own straight-line boundary,  
8   # same formula as the very first neural networks tutorial:  $w1*x1 + w2*x2 + b = 0$ , solved for  $x2$   
9   y_line <- -(best_run$W1[i, 1] * x_seq + best_run$b1[i]) / best_run$W1[i, 2]  
10  lines(x_seq, y_line, col = i + 1, lwd = 2)  
11 }  
12 legend("topright", legend = c("hidden unit 1's line", "hidden unit 2's line"),  
13       col = c(2, 3), lty = 1, lwd = 2)
```

The two lines the network actually learned



# It works! (3/3)

- Each hidden unit really did learn its own straight line which are the same *kind* of “two lines, combined” solution as the recap slide’s hand-picked weights.

Note, there’s more than one valid way to split XOR into two lines and the network will find its own pair from a random start, using nothing but backpropagated error.

- How is each line’s equation actually computed? Each hidden unit is just a perceptron, so it has the exact same straight-line boundary formula from the very first neural networks tutorial: setting  $w_{i1}x_1 + w_{i2}x_2 + b_i = 0$  and solving for  $x_2$  gives  $x_2 = -(w_{i1}x_1 + b_i)/w_{i2}$  — that’s precisely `y_line <- -(best_run$W1[i, 1] * x_seq + best_run$b1[i]) / best_run$W1[i, 2]` in the code above, just applied to row  $i$  of `W1` for each hidden unit in turn
- The output unit’s job was just to combine those two lines correctly (see also ‘*Why does the output unit’s job turn out to be easy?*’), which is exactly what the delta rule, applied at the output layer, trained it to do

# The bigger picture, again

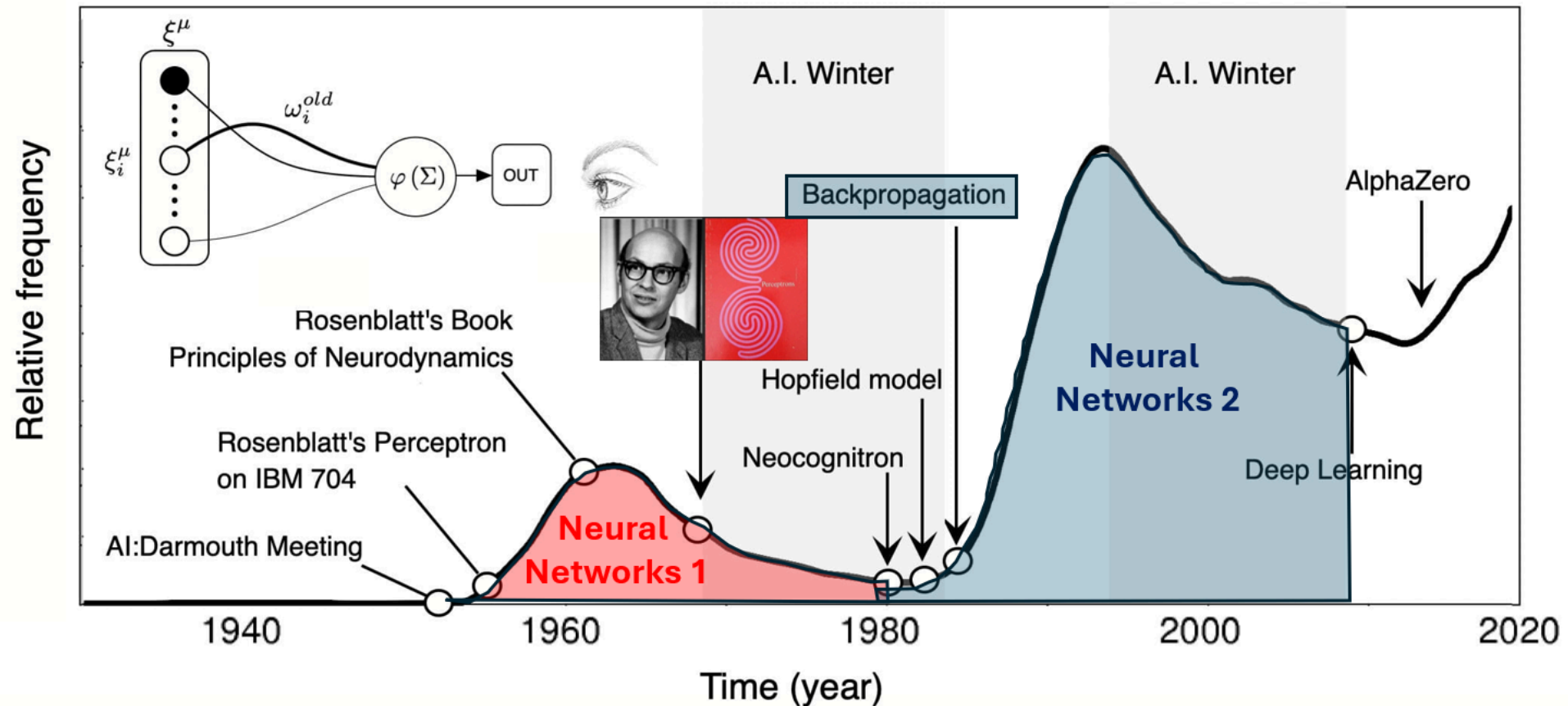
- Previously: today's mechanism (nudge weights to reduce error) is the same one behind large language models and modern deep learning, just at enormous scale
- What we added today (a **hidden layer**) is literally what “deep” in “deep learning” refers to: stack more layers, and the same backpropagation idea (chain rule, applied once per layer) still trains them, it just has more hops or steps to make on the way back
- A real network might have dozens of layers and billions of weights instead of 2 hidden units and 9 parameters (including weights and biases), but every one of those weights and biases still gets updated by exactly this mechanism: a local error signal, scaled by a local derivative, passed backward one layer at a time

# Final remarks

# Summary

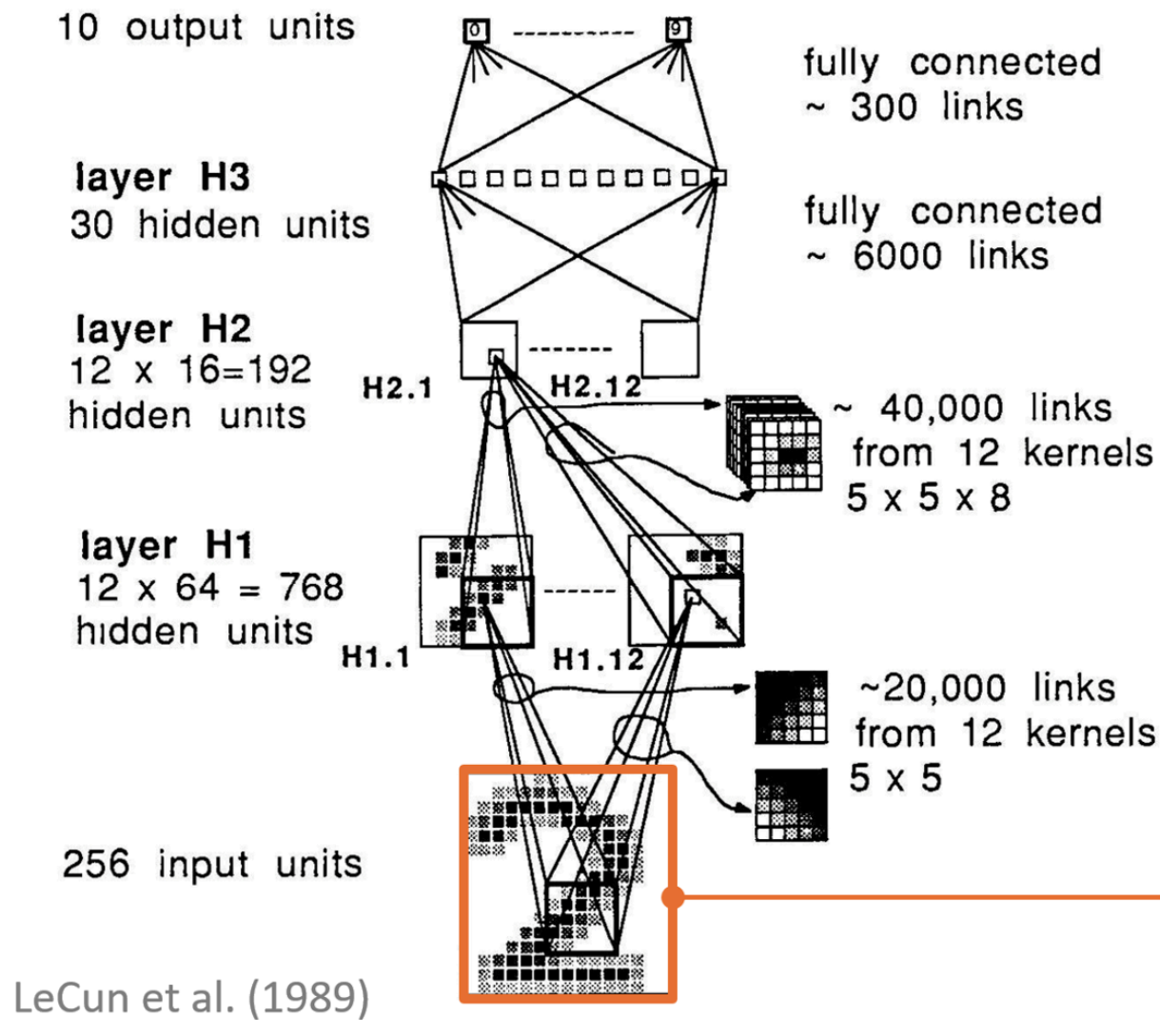
- A **hidden layer** is several ordinary perceptrons sitting between the inputs and the output unit. It's no new kind of unit, just a new arrangement
- Hand-picked weights prove a 2-hidden-unit network *can* represent XOR (**OR**, **NAND**, and AND, combined); the open question was always how to *find* such weights automatically
- **Backpropagation** answers that: the delta rule, applied at the output layer exactly as before, then applied again at the hidden layer using an error signal sent backward through the output weights
- Training from a random start occasionally gets stuck in a bad local minimum, but trying a few random restarts and keeping the best is the standard, practical fix
- Zoom out: this is still the same core mechanism from previously (i.e., nudge weights to reduce error) just with one more layer, and one more step or hop for the error signal to travel backward through

# Final Remarks: The springs and winters of AI development



[https://x.com/ricard\\_sole/status/1497306702181548039](https://x.com/ricard_sole/status/1497306702181548039)

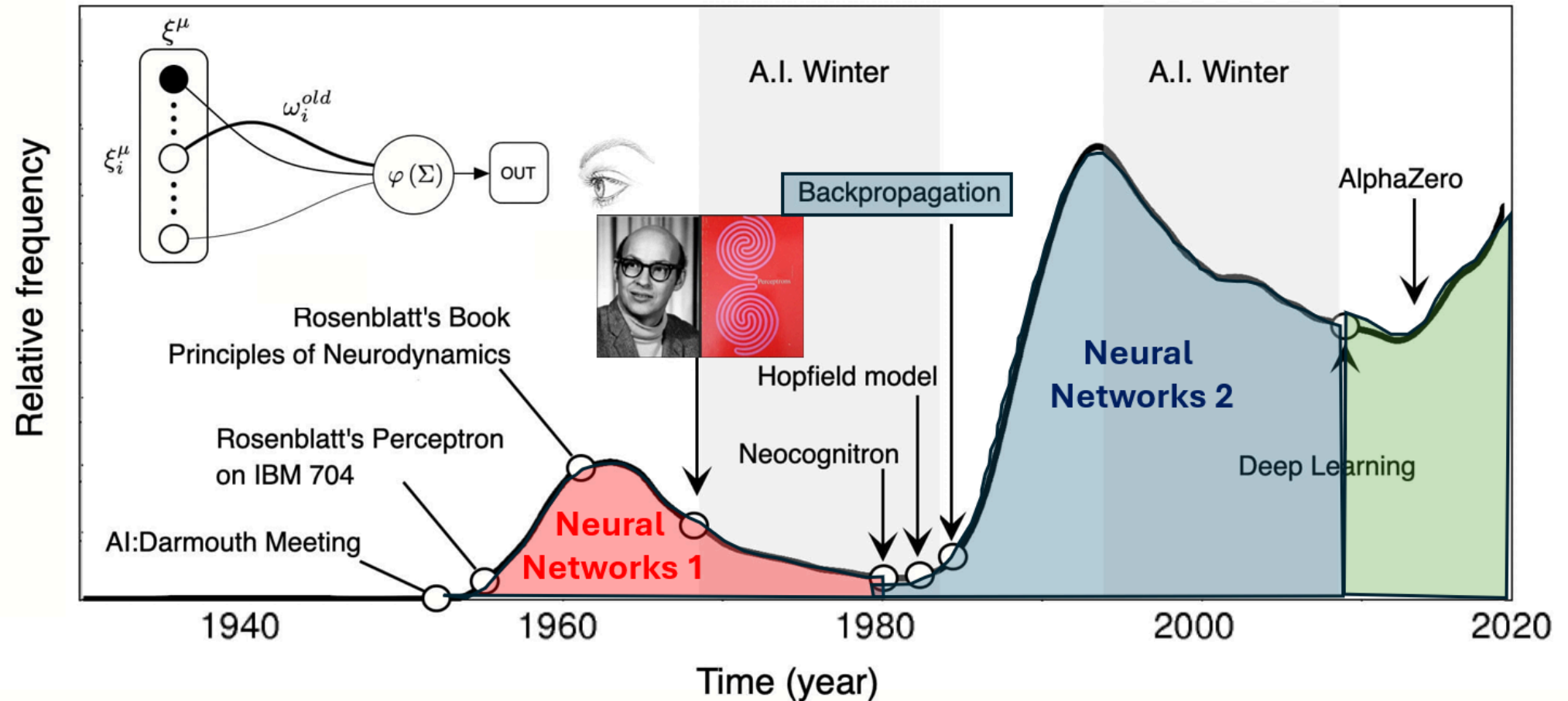
# Final Remarks: The springs and winters of AI development



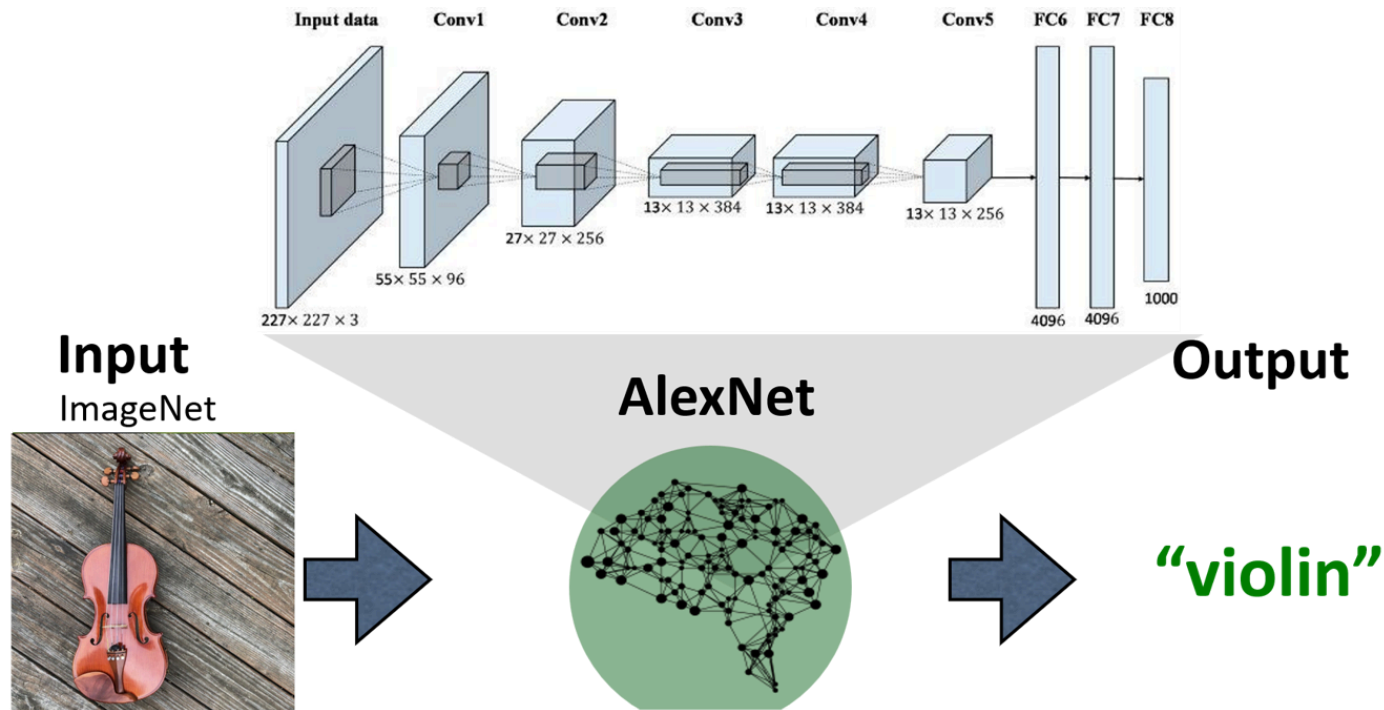
## Example Inputs

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 5 | 7 | 7 | 7 | 2 | 8 | 6 | 4 |
| 4 | 9 | 5 | 4 | 8 | 5 | 6 | 8 |
| 4 | 3 | 7 | 7 | 1 | 2 | 3 | 8 |
| 2 | 4 | 4 | 6 | 8 | 2 | 3 | 3 |
| 8 | 6 | 3 | 3 | 4 | 4 | 6 | 8 |
| 8 | 6 | 3 | 6 | 2 | 3 | 5 | 5 |

# Final Remarks: The springs and winters of AI development



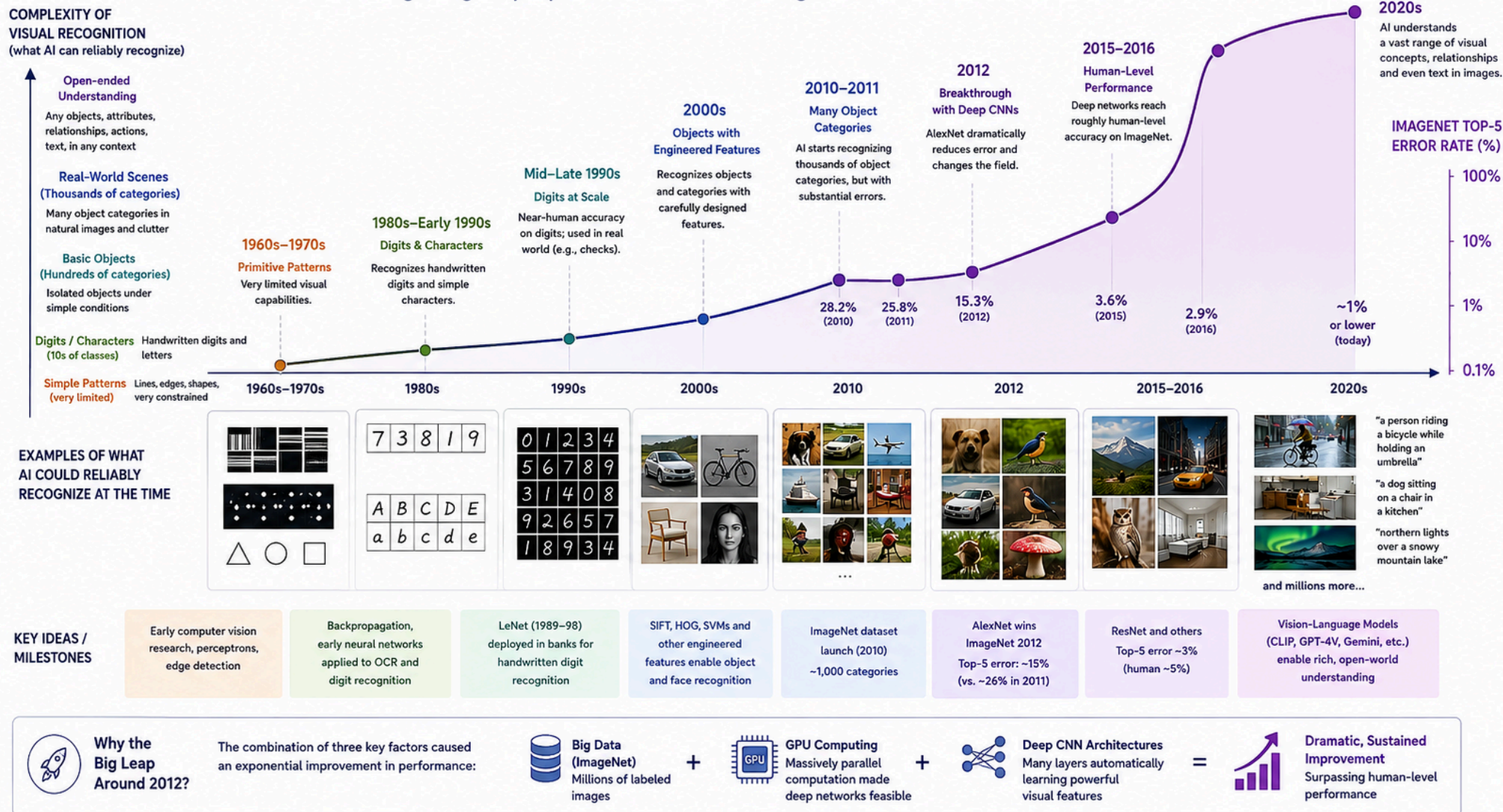
# Final Remarks: The springs and winters of AI development



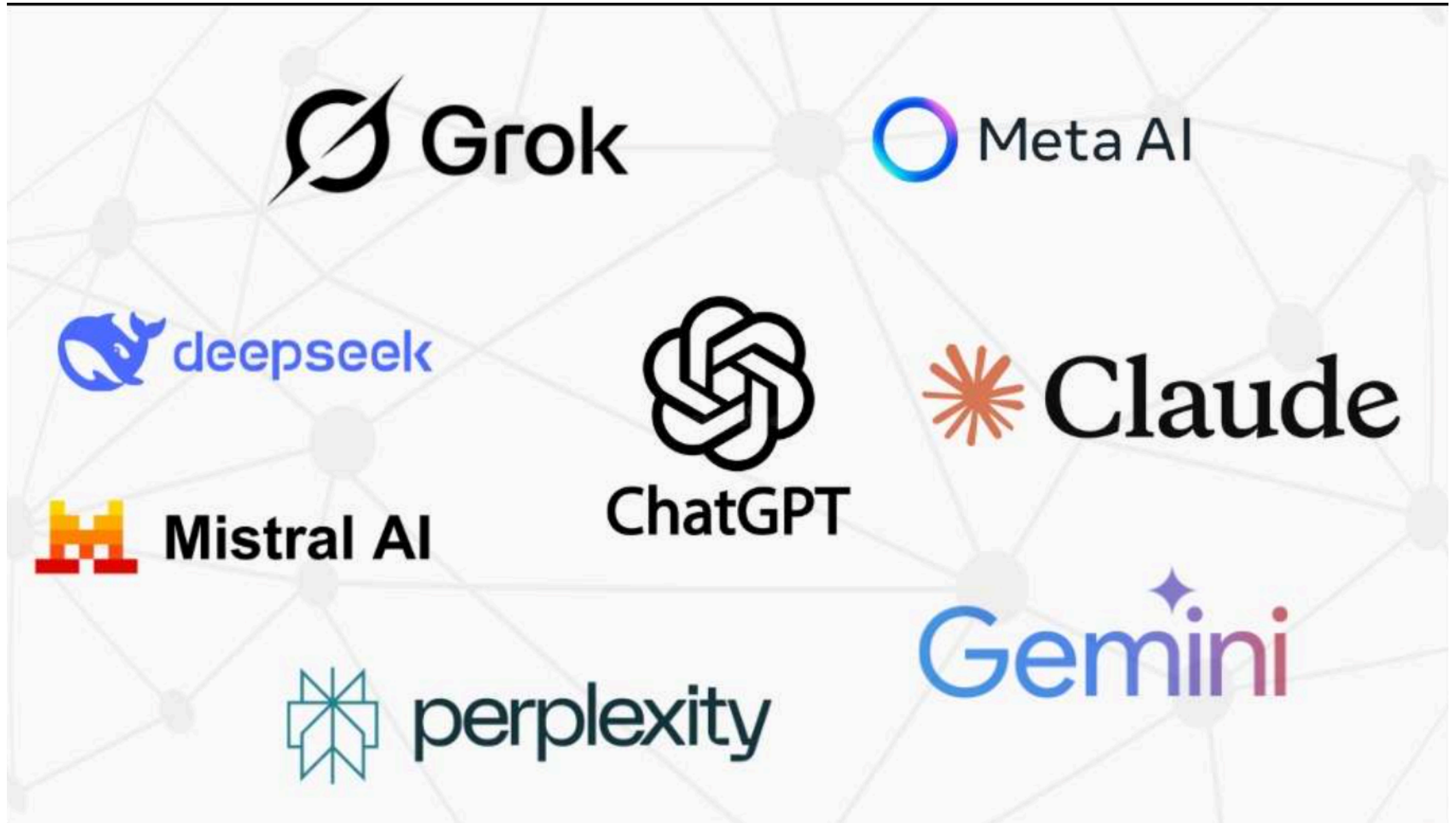
# Final Remarks: The springs and winters of AI development

## The Explosive Growth of AI Visual Recognition

From recognizing simple patterns to understanding the full richness of the visual world



# Final Remarks: The springs and winters of AI development



# Questions?